

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5

7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I VENNA MEENAKSHI REDDY(192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VENNA MEENAKSHI REDDY

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

		solve the problem			
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

Recommended Strategy: Heap File Organization with B+ Tree Indexing

An e-commerce company has to store around 10 million product records. Since products are frequently added, modified, and removed, the file organization should support efficient insertion and retrieval. A suitable solution is to store the records using a Heap File Organization and create B+ Tree indexes on frequently searched attributes.

1. Heap File Organization

In a heap file, records are stored in no particular order. A new product can be placed in any block having free space.

For example:

Product File

```
|  
+-- Block 1 → Product records  
+-- Block 2 → Product records  
+-- Block 3 → Product records  
+-- ...  
+-- Block N → Product records
```

2. Why Heap File?

Heap organization is suitable because:

- Products are continuously inserted.
- No sorting is required during insertion.
- New records can be inserted into available free space.
- Deleted space can be reused.
- It is simple to maintain.
- It avoids the high cost of maintaining physical sorted order.

3. B+ Tree Index

Searching through 10 million records sequentially would be expensive. Therefore, B+ Tree indexes should be created.

Possible indexes:

B+ Tree → ProductID

B+ Tree → Category

B+ Tree → Price

For example, a B+ Tree on ProductID allows the system to quickly locate a particular product.

4. Range Query Support

An e-commerce system commonly performs queries such as:

```
SELECT *
```

```
FROM Product
```

```
WHERE Price BETWEEN 1000 AND 5000;
```

A B+ Tree on Price can efficiently locate the first price and then scan the linked leaf nodes until the required range is completed.

5. Cost Analysis

Operation	Heap File	Heap + B+ Tree
Insert	Very fast	Fast
Exact search	$O(N)$	$O(\log N)$ approximately
Range search	$O(N)$	Efficient
Delete	Fast	Fast–Moderate
Storage cost	Low	Higher
Maintenance	Low	Moderate

Without indexing, a product search may require scanning millions of records. A B+ Tree significantly reduces the number of disk accesses.

6. Trade-Off

The main disadvantage is that B+ Tree indexes require additional:

- Disk space
- Memory
- Index maintenance during insertion and deletion

However, for a large e-commerce system, the retrieval improvement is much more valuable than the additional storage cost.

Conclusion

For 10 million products, the recommended design is Heap File Organization with B+ Tree secondary indexes on ProductID, Category, and Price. This provides fast insertion, efficient exact and range searching, low storage complexity, and good scalability.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli X + v

mysql> -- =====
mysql> -- Q1: E-COMMERCE PRODUCT DATABASE
mysql> -- =====
mysql> CREATE DATABASE EcommerceDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE EcommerceDB;
Database changed
mysql>
mysql> -- Create Product table
mysql> CREATE TABLE Products (
  -> ProductID INT PRIMARY KEY,
  -> ProductName VARCHAR(100) NOT NULL,
  -> Category VARCHAR(50),
  -> Price DECIMAL(10,2),
  -> Stock INT,
  -> Brand VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- Insert sample records
mysql> INSERT INTO Products
  -> (ProductID, ProductName, Category, Price, Stock, Brand)
  -> VALUES
  -> (101, 'Laptop', 'Electronics', 55000.00, 20, 'Dell'),
  -> (102, 'Smartphone', 'Electronics', 25000.00, 50, 'Samsung'),
  -> (103, 'Headphones', 'Electronics', 2500.00, 100, 'Sony'),
  -> (104, 'Shoes', 'Fashion', 3000.00, 75, 'Nike'),
  -> (105, 'Watch', 'Fashion', 5000.00, 30, 'Titan'),
  -> (106, 'Keyboard', 'Electronics', 1500.00, 60, 'HP'),
  -> (107, 'Backpack', 'Fashion', 1800.00, 40, 'Wildcraft'),
  -> (108, 'Tablet', 'Electronics', 22000.00, 25, 'Lenovo');
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Display all products
mysql> SELECT * FROM Products;
+-----+-----+-----+-----+-----+-----+
| ProductID | ProductName | Category | Price | Stock | Brand |
+-----+-----+-----+-----+-----+-----+
| 101 | Laptop | Electronics | 55000.00 | 20 | Dell |
| 102 | Smartphone | Electronics | 25000.00 | 50 | Samsung |
| 103 | Headphones | Electronics | 2500.00 | 100 | Sony |
| 104 | Shoes | Fashion | 3000.00 | 75 | Nike |
| 105 | Watch | Fashion | 5000.00 | 30 | Titan |
| 106 | Keyboard | Electronics | 1500.00 | 60 | HP |
| 107 | Backpack | Fashion | 1800.00 | 40 | Wildcraft |
| 108 | Tablet | Electronics | 22000.00 | 25 | Lenovo |
+-----+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql>
mysql> -- Create B+ Tree equivalent indexes
mysql> CREATE INDEX idx_product_category
  -> ON Products(Category);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> CREATE INDEX idx_product_price
  -> ON Products(Price);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli X + v

mysql>
mysql> -- Search product by ProductID
mysql> SELECT *
  -> FROM Products
  -> WHERE ProductID = 102;
+-----+-----+-----+-----+-----+-----+
| ProductID | ProductName | Category | Price | Stock | Brand |
+-----+-----+-----+-----+-----+-----+
| 102 | Smartphone | Electronics | 25000.00 | 50 | Samsung |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Search products by category
mysql> SELECT *
  -> FROM Products
  -> WHERE Category = 'Electronics';
+-----+-----+-----+-----+-----+-----+
| ProductID | ProductName | Category | Price | Stock | Brand |
+-----+-----+-----+-----+-----+-----+
| 101 | Laptop | Electronics | 55000.00 | 20 | Dell |
| 102 | Smartphone | Electronics | 25000.00 | 50 | Samsung |
| 103 | Headphones | Electronics | 2500.00 | 100 | Sony |
| 106 | Keyboard | Electronics | 1500.00 | 60 | HP |
| 108 | Tablet | Electronics | 22000.00 | 25 | Lenovo |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- Range search by price
mysql> SELECT *
  -> FROM Products
  -> WHERE Price BETWEEN 2000 AND 30000;
+-----+-----+-----+-----+-----+-----+
| ProductID | ProductName | Category | Price | Stock | Brand |
+-----+-----+-----+-----+-----+-----+
| 103 | Headphones | Electronics | 2500.00 | 100 | Sony |
| 104 | Shoes | Fashion | 3000.00 | 75 | Nike |
| 105 | Watch | Fashion | 5000.00 | 30 | Titan |
| 108 | Tablet | Electronics | 22000.00 | 25 | Lenovo |
| 102 | Smartphone | Electronics | 25000.00 | 50 | Samsung |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>
mysql> -- Search using category and price
mysql> SELECT *
  -> FROM Products
  -> WHERE Category = 'Electronics'
  -> AND Price BETWEEN 10000 AND 60000;
+-----+-----+-----+-----+-----+-----+
| ProductID | ProductName | Category | Price | Stock | Brand |
+-----+-----+-----+-----+-----+-----+
| 101 | Laptop | Electronics | 55000.00 | 20 | Dell |
| 102 | Smartphone | Electronics | 25000.00 | 50 | Samsung |
| 108 | Tablet | Electronics | 22000.00 | 25 | Lenovo |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Check index usage
mysql> EXPLAIN
  -> SELECT *
  -> FROM Products
  -> WHERE Category = 'Electronics'
  -> AND Price BETWEEN 10000 AND 60000;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Products | NULL | ref | idx_product_category,idx_product_price | idx_product_category | 203 | const | 5 | 37.50 | Using where |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
```

2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Recommended Strategy: Composite B+ Tree Index

A banking system must quickly retrieve transactions using both Account Number and Transaction Date. The most appropriate solution is a composite B+ Tree index on (AccountNumber, TransactionDate).

1. Requirement

Consider a query:

```
SELECT *FROM Transactions
```

```
WHERE AccountNumber = 10525
```

```
AND TransactionDate BETWEEN '2026-01-01' AND '2026-03-31';
```

The system must first identify the account and then retrieve transactions within the specified date range.

2. Composite Index

Create:

B+ Tree Index (AccountNumber, TransactionDate)

The index is ordered first by AccountNumber and then by TransactionDate.

Example:

Account 1001 → Jan → Feb → Mar → Apr

Account 1002 → Jan → Feb → Mar → Apr

Account 1003 → Jan → Feb → Mar → Apr

3. Search Process

For a particular account:

1. Traverse the B+ Tree to locate the account.
2. Locate the first required date.
3. Follow the leaf-node links.
4. Retrieve transactions until the ending date is reached.

4. Why B+ Tree?

B+ Trees are ideal for database systems because:

- They have high fan-out.
- Tree height remains small.

- Searching requires approximately $O(\log N)$ index traversal.
- Leaf nodes are linked.
- They efficiently support range queries.
- They work well with disk-based storage.

5. Why Not Hashing?

A hash index is excellent for:

AccountNumber = 10525

but poor for:

TransactionDate BETWEEN Date1 AND Date2

because hash buckets do not maintain sorted order.

6. Alternative Index

A separate index on:

TransactionDate

can also be maintained if the banking system frequently searches transactions across all accounts for a date range.

7. Performance Comparison

Feature	Hash Index	B+ Tree
Account exact search	Excellent	Excellent
Date range	Poor	Excellent
Account + date range	Moderate	Excellent
Ordered traversal	Poor	Excellent
Dynamic updates	Good	Good

Conclusion

A composite B+ Tree index on (AccountNumber, TransactionDate) is the best solution because it efficiently supports account-specific date-range queries and minimizes disk I/O.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli × + ∨

mysql> -- =====
mysql> -- Q2: BANKING TRANSACTION DATABASE
mysql> -- =====
mysql>
mysql> CREATE DATABASE BankingDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE BankingDB;
Database changed
mysql>
mysql> CREATE TABLE Transactions (
  -> TransactionID INT PRIMARY KEY,
  -> AccountNumber INT NOT NULL,
  -> TransactionDate DATE NOT NULL,
  -> TransactionType VARCHAR(20),
  -> Amount DECIMAL(10,2),
  -> Description VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- Insert transactions
mysql> INSERT INTO Transactions
  -> VALUES
  -> (1001, 101, '2026-01-05', 'Deposit', 5000.00, 'Salary'),
  -> (1002, 101, '2026-01-15', 'Withdrawal', 1000.00, 'ATM'),
  -> (1003, 101, '2026-02-10', 'Deposit', 3000.00, 'Cash Deposit'),
  -> (1004, 101, '2026-02-25', 'Withdrawal', 500.00, 'Shopping'),
  -> (1005, 101, '2026-03-05', 'Deposit', 4000.00, 'Salary'),
  -> (1006, 102, '2026-01-10', 'Deposit', 6000.00, 'Salary'),
  -> (1007, 102, '2026-02-15', 'Withdrawal', 1500.00, 'ATM'),
  -> (1008, 103, '2026-02-05', 'Deposit', 8000.00, 'Salary'),
  -> (1009, 103, '2026-03-10', 'Withdrawal', 2000.00, 'Rent');
Query OK, 9 rows affected (0.01 sec)
Records: 9 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Display records
mysql> SELECT * FROM Transactions;
+-----+-----+-----+-----+-----+-----+
| TransactionID | AccountNumber | TransactionDate | TransactionType | Amount | Description |
+-----+-----+-----+-----+-----+-----+
| 1001 | 101 | 2026-01-05 | Deposit | 5000.00 | Salary |
| 1002 | 101 | 2026-01-15 | Withdrawal | 1000.00 | ATM |
| 1003 | 101 | 2026-02-10 | Deposit | 3000.00 | Cash Deposit |
| 1004 | 101 | 2026-02-25 | Withdrawal | 500.00 | Shopping |
| 1005 | 101 | 2026-03-05 | Deposit | 4000.00 | Salary |
| 1006 | 102 | 2026-01-10 | Deposit | 6000.00 | Salary |
| 1007 | 102 | 2026-02-15 | Withdrawal | 1500.00 | ATM |
| 1008 | 103 | 2026-02-05 | Deposit | 8000.00 | Salary |
| 1009 | 103 | 2026-03-10 | Withdrawal | 2000.00 | Rent |
+-----+-----+-----+-----+-----+-----+
9 rows in set (0.00 sec)

mysql>
mysql> -- Composite index
mysql> CREATE INDEX idx_account_date
  -> ON Transactions(AccountNumber, TransactionDate);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Search by account number
mysql> SELECT *
  -> FROM Transactions
  -> WHERE AccountNumber = 101;
+-----+-----+-----+-----+-----+-----+
| TransactionID | AccountNumber | TransactionDate | TransactionType | Amount | Description |
+-----+-----+-----+-----+-----+-----+
| 1001 | 101 | 2026-01-05 | Deposit | 5000.00 | Salary |
| 1002 | 101 | 2026-01-15 | Withdrawal | 1000.00 | ATM |
| 1003 | 101 | 2026-02-10 | Deposit | 3000.00 | Cash Deposit |
| 1004 | 101 | 2026-02-25 | Withdrawal | 500.00 | Shopping |
| 1005 | 101 | 2026-03-05 | Deposit | 4000.00 | Salary |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- Search by account number and date range
mysql> SELECT *
  -> FROM Transactions
  -> WHERE AccountNumber = 101
  -> AND TransactionDate BETWEEN '2026-01-01'
  -> AND '2026-03-31';
+-----+-----+-----+-----+-----+-----+
| TransactionID | AccountNumber | TransactionDate | TransactionType | Amount | Description |
+-----+-----+-----+-----+-----+-----+
| 1001 | 101 | 2026-01-05 | Deposit | 5000.00 | Salary |
| 1002 | 101 | 2026-01-15 | Withdrawal | 1000.00 | ATM |
| 1003 | 101 | 2026-02-10 | Deposit | 3000.00 | Cash Deposit |
| 1004 | 101 | 2026-02-25 | Withdrawal | 500.00 | Shopping |
| 1005 | 101 | 2026-03-05 | Deposit | 4000.00 | Salary |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- Check index usage
mysql> EXPLAIN
  -> SELECT *
  -> FROM Transactions
  -> WHERE AccountNumber = 101
  -> AND TransactionDate BETWEEN '2026-01-01'
  -> AND '2026-03-31';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Transactions | NULL | range | idx_account_date | idx_account_date | 7 | NULL | 5 | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql> -- Sort transactions by date
mysql> SELECT *
  -> FROM Transactions
  -> WHERE AccountNumber = 101
  -> ORDER BY TransactionDate;
+-----+-----+-----+-----+-----+-----+
| TransactionID | AccountNumber | TransactionDate | TransactionType | Amount | Description |
+-----+-----+-----+-----+-----+-----+
| 1001 | 101 | 2026-01-05 | Deposit | 5000.00 | Salary |
| 1002 | 101 | 2026-01-15 | Withdrawal | 1000.00 | ATM |
| 1003 | 101 | 2026-02-10 | Deposit | 3000.00 | Cash Deposit |
| 1004 | 101 | 2026-02-25 | Withdrawal | 500.00 | Shopping |
| 1005 | 101 | 2026-03-05 | Deposit | 4000.00 | Salary |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> |
```

3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Recommended Strategy: Multi-Level B+ Tree Indexing

A healthcare database must provide quick access to patient records using two different attributes:

- PatientID
- DoctorName

Therefore, two separate secondary B+ Tree indexes should be maintained.

1. PatientID Index

Create:

B+ Tree

|

PatientID

|

Patient Record Pointer

Since PatientID is normally unique, each key can point to one patient record.

2. DoctorName Index

Create another B+ Tree:

B+ Tree

|

DoctorName

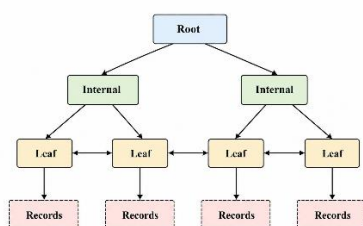
|

Multiple Patient Record Pointers

DoctorName may not be unique because one doctor can treat many patients.

3. Multi-Level Structure

A B+ Tree can contain several levels:



The root directs the search toward the appropriate internal node and finally to the leaf.

4. PatientID Query

Example:

```
SELECT *FROM Patient WHERE PatientID = 50125;
```

The PatientID B+ Tree quickly identifies the required record.

5. Doctor Query

Example:

```
SELECT *FROM Patient WHERE DoctorName = 'Dr. Rao';
```

The DoctorName index locates all leaf entries associated with Dr. Rao.

6. Advantages

- Fast PatientID retrieval.
- Fast doctor-based retrieval.
- Supports range queries if required.
- Reduces disk accesses.
- Suitable for millions of patient records.
- Indexes can grow dynamically.

7. Disadvantages

Maintaining two indexes creates:

- Additional storage requirements.
- Extra insertion overhead.
- Extra deletion/update overhead.

However, healthcare systems prioritize fast retrieval, so this overhead is justified.

Conclusion

A multi-level B+ Tree on PatientID and a separate secondary B+ Tree on DoctorName provides multiple efficient access paths and is suitable for a large healthcare database.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli
mysql> -- Q3: HEALTHCARE DATABASE
mysql> CREATE DATABASE HealthcareDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE HealthcareDB;
Database changed
mysql> CREATE TABLE Patients (
  -> PatientID INT PRIMARY KEY,
  -> PatientName VARCHAR(100),
  -> Age INT,
  -> Gender VARCHAR(10),
  -> DoctorName VARCHAR(100),
  -> Disease VARCHAR(100),
  -> AdmissionDate DATE
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> -- Insert patient records
mysql> INSERT INTO Patients
  -> VALUES
  -> (1, 'Rahul', 25, 'Male', 'Dr. Rao', 'Fever', '2026-01-10'),
  -> (2, 'Anita', 32, 'Female', 'Dr. Sharma', 'Diabetes', '2026-01-12'),
  -> (3, 'Kiran', 45, 'Male', 'Dr. Rao', 'Heart Disease', '2026-01-15'),
  -> (4, 'Priya', 28, 'Female', 'Dr. Kumar', 'Fever', '2026-02-01'),
  -> (5, 'Arjun', 50, 'Male', 'Dr. Rao', 'Blood Pressure', '2026-02-05'),
  -> (6, 'Sneha', 30, 'Female', 'Dr. Sharma', 'Asthma', '2026-02-10'),
  -> (7, 'Vijay', 40, 'Male', 'Dr. Kumar', 'Diabetes', '2026-02-15');
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql> -- Display records
mysql> SELECT * FROM Patients;
+-----+-----+-----+-----+-----+-----+-----+
| PatientID | PatientName | Age | Gender | DoctorName | Disease | AdmissionDate |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Rahul | 25 | Male | Dr. Rao | Fever | 2026-01-10 |
| 2 | Anita | 32 | Female | Dr. Sharma | Diabetes | 2026-01-12 |
| 3 | Kiran | 45 | Male | Dr. Rao | Heart Disease | 2026-01-15 |
| 4 | Priya | 28 | Female | Dr. Kumar | Fever | 2026-02-01 |
| 5 | Arjun | 50 | Male | Dr. Rao | Blood Pressure | 2026-02-05 |
| 6 | Sneha | 30 | Female | Dr. Sharma | Asthma | 2026-02-10 |
| 7 | Vijay | 40 | Male | Dr. Kumar | Diabetes | 2026-02-15 |
+-----+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)

mysql> -- Index on PatientID
mysql> CREATE INDEX idx_patient_id
  -> ON Patients(PatientID);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> -- Index on DoctorName
mysql> CREATE INDEX idx_doctor_name
  -> ON Patients(DoctorName);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> -- Search by PatientID
mysql> SELECT *
  -> FROM Patients
  -> WHERE PatientID = 3;
+-----+-----+-----+-----+-----+-----+-----+
| PatientID | PatientName | Age | Gender | DoctorName | Disease | AdmissionDate |
+-----+-----+-----+-----+-----+-----+-----+
| 3 | Kiran | 45 | Male | Dr. Rao | Heart Disease | 2026-01-15 |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> -- Search by DoctorName
mysql> SELECT *
  -> FROM Patients
  -> WHERE DoctorName = 'Dr. Rao';
+-----+-----+-----+-----+-----+-----+-----+
| PatientID | PatientName | Age | Gender | DoctorName | Disease | AdmissionDate |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Rahul | 25 | Male | Dr. Rao | Fever | 2026-01-10 |
| 3 | Kiran | 45 | Male | Dr. Rao | Heart Disease | 2026-01-15 |
| 5 | Arjun | 50 | Male | Dr. Rao | Blood Pressure | 2026-02-05 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> -- Search patients treated by a doctor
mysql> SELECT PatientID, PatientName, Disease
  -> FROM Patients
  -> WHERE DoctorName = 'Dr. Rao';
+-----+-----+-----+
| PatientID | PatientName | Disease |
+-----+-----+-----+
| 1 | Rahul | Fever |
| 3 | Kiran | Heart Disease |
| 5 | Arjun | Blood Pressure |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> -- Check index usage
mysql> EXPLAIN
  -> SELECT *
  -> FROM Patients
  -> WHERE DoctorName = 'Dr. Rao';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Patients | NULL | ref | idx_doctor_name | idx_doctor_name | 403 | const | 3 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> |
```

4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Recommended Choice: B+ Tree

A logistics company frequently inserts and deletes shipment records. It also needs to search shipments based on shipment ID, date, location, and delivery status. Between B-Tree and B+ Tree, B+ Tree is more suitable.

1. B-Tree

In a B-Tree, both internal and leaf nodes can contain:

- Search keys
- Record pointers

Therefore, actual data may be found at different levels.

2. B+ Tree

In a B+ Tree:

- Internal nodes contain keys and child pointers.
- Actual record pointers are stored at leaf nodes.
- Leaf nodes are linked sequentially.

3. Insertion

When a shipment is inserted:

1. Locate the appropriate leaf.
2. Insert the key.
3. If the leaf becomes full, split it.
4. Promote the separator key.
5. Continue upward if necessary.

4. Deletion

When a shipment is deleted:

1. Locate the leaf.
2. Delete the key.
3. If the node becomes underfilled, redistribute entries or merge nodes.
4. Update the parent.

5. Comparison

Feature	B-Tree	B+ Tree
Search	Good	Excellent
Insert	Good	Excellent
Delete	Good	Excellent
Range queries	Good	Excellent
Sequential access	Moderate	Excellent
Fan-out	Lower	Higher
Disk storage	Good	Excellent

6. Why B+ Tree?

Logistics systems commonly require queries such as:

Find all shipments from 1 August to 15 August

B+ Tree can locate the first record and then traverse linked leaf nodes.

Higher fan-out also means fewer tree levels, resulting in fewer disk accesses.

Conclusion

B+ Tree is superior for the logistics system because it provides efficient dynamic insertion and deletion, high fan-out, excellent range queries, and fast sequential access.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli
mysql> --=====
mysql> -- Q4: LOGISTICS DATABASE
mysql> --=====
mysql> CREATE DATABASE LogisticsDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE LogisticsDB;
Database changed
mysql> CREATE TABLE Shipments (
  ->   ShipmentID INT PRIMARY KEY,
  ->   CustomerName VARCHAR(100),
  ->   SourceLocation VARCHAR(50),
  ->   Destination VARCHAR(50),
  ->   ShipmentDate DATE,
  ->   Status VARCHAR(30)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> -- Insert shipment records
mysql> INSERT INTO Shipments
  -> VALUES
  -> (1001, 'Ravi', 'Hyderabad', 'Chennai', '2026-01-05', 'Delivered'),
  -> (1002, 'Anil', 'Bangalore', 'Hyderabad', '2026-01-10', 'In Transit'),
  -> (1003, 'Priya', 'Delhi', 'Mumbai', '2026-01-15', 'Delivered'),
  -> (1004, 'Kiran', 'Chennai', 'Pune', '2026-02-01', 'Pending'),
  -> (1005, 'Sneha', 'Mumbai', 'Delhi', '2026-02-10', 'In Transit'),
  -> (1006, 'Arjun', 'Pune', 'Hyderabad', '2026-02-20', 'Delivered');
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> -- Create indexes
mysql> CREATE INDEX idx_shipment_id
  -> ON Shipments(ShipmentID);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_shipment_date
  -> ON Shipments(ShipmentDate);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> -- Search shipment
mysql> SELECT *
  -> FROM Shipments
  -> WHERE ShipmentID = 1003;
+-----+-----+-----+-----+-----+-----+
| ShipmentID | CustomerName | SourceLocation | Destination | ShipmentDate | Status |
+-----+-----+-----+-----+-----+-----+
| 1003 | Priya | Delhi | Mumbai | 2026-01-15 | Delivered |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> -- Range search
mysql> SELECT *
  -> FROM Shipments
  -> WHERE ShipmentDate BETWEEN '2026-01-01'
  -> AND '2026-02-28';
+-----+-----+-----+-----+-----+-----+
| ShipmentID | CustomerName | SourceLocation | Destination | ShipmentDate | Status |
+-----+-----+-----+-----+-----+-----+
| 1001 | Ravi | Hyderabad | Chennai | 2026-01-05 | Delivered |
| 1002 | Anil | Bangalore | Hyderabad | 2026-01-10 | In Transit |
| 1003 | Priya | Delhi | Mumbai | 2026-01-15 | Delivered |
| 1004 | Kiran | Chennai | Pune | 2026-02-01 | Pending |
| 1005 | Sneha | Mumbai | Delhi | 2026-02-10 | In Transit |
| 1006 | Arjun | Pune | Hyderabad | 2026-02-20 | Delivered |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

```

mysql>
mysql> -- Insert new shipment
mysql> INSERT INTO Shipments
mysql>   -> VALUES
mysql>   -> (1007, 'Meghana', 'Hyderabad', 'Bangalore',
mysql>   -> '2026-03-01', 'Pending');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Update shipment
mysql> UPDATE Shipments
mysql>   -> SET Status = 'Delivered'
mysql>   -> WHERE ShipmentID = 1007;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> -- Delete shipment
mysql> DELETE FROM Shipments
mysql>   -> WHERE ShipmentID = 1007;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Check execution plan
mysql> EXPLAIN
mysql>   -> SELECT *
mysql>   -> FROM Shipments
mysql>   -> WHERE ShipmentDate BETWEEN '2026-01-01'
mysql>   -> AND '2026-02-28';

```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Shipments	NULL	range	idx_shipment_date	idx_shipment_date	4	NULL	6	100.00	Using index condition

```

1 row in set, 1 warning (0.00 sec)

mysql> |

```

5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Recommended Strategy: Extendible Hashing

An HR system containing 5,000 employees can use hashing for fast EmployeeID-based retrieval. Two possible methods are Static Hashing and Extendible Hashing.

Static Hashing

Suppose we create 500 buckets:

$$h(\text{EmployeeID}) = \text{EmployeeID} \bmod 500$$

Each employee is assigned to a bucket based on the hash function.

Advantages

- Simple.
- Fast exact search.
- Low implementation complexity.
- Good when the number of records is fixed.

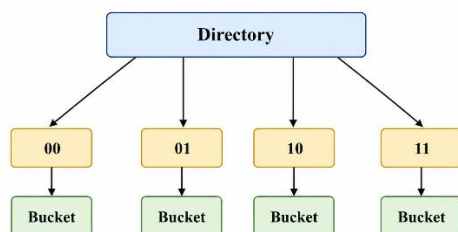
Disadvantages

If the company grows from 5,000 to 10,000 employees:

- Existing buckets may become overloaded.
- Overflow chains may develop.
- Searching through overflow records reduces performance.
- Reorganization can be expensive.

Extendible Hashing

Extendible hashing uses a directory and dynamically growing buckets.



When a bucket becomes full, it can be split.

Advantages

- Dynamic growth.
- Less overflow.

- Efficient exact searches.
- Does not require rebuilding the entire file.
- Suitable for changing employee populations.

Comparison

Feature	Static Hashing	Extendible Hashing
Implementation	Simple	More complex
Exact search	Excellent	Excellent
Growth	Poor	Excellent
Overflow	Higher	Lower
Dynamic resizing	Difficult	Easy
Scalability	Moderate	High

Conclusion

For exactly 5,000 relatively stable employees, static hashing is sufficient. However, extendible hashing is the better long-term solution because employee records may continuously increase or decrease.

SQL IMPLEMENTATION

```

MySQL 8.0 Command Line Cli
mysql> -- =====
mysql> -- Q5: HR HASHING DATABASE
mysql> -- =====
mysql> CREATE DATABASE HRDB;
ERROR 1007 (HY000): Can't create database 'hrdb'; database exists
mysql> USE HRDB;
Database changed
mysql> CREATE TABLE Employees (
  -- EmployeeID INT PRIMARY KEY,
  -- EmployeeName VARCHAR(100),
  -- Department VARCHAR(50),
  -- Salary DECIMAL(10,2),
  -- HashBucket INT
  -- );
Query OK, 0 rows affected (0.04 sec)

mysql> -- Insert employee records
mysql> INSERT INTO Employees
mysql> VALUES
  -- (1001, 'Ravi', 'IT', 45000, MOD(1001, 10)),
  -- (1002, 'Anil', 'HR', 40000, MOD(1002, 10)),
  -- (1003, 'Priya', 'Finance', 50000, MOD(1003, 10)),
  -- (1004, 'Kiran', 'IT', 55000, MOD(1004, 10)),
  -- (1005, 'Sneha', 'HR', 42000, MOD(1005, 10)),
  -- (1006, 'Arjun', 'Sales', 38000, MOD(1006, 10)),
  -- (1007, 'Vijay', 'IT', 60000, MOD(1007, 10)),
  -- (1008, 'Pooja', 'Finance', 52000, MOD(1008, 10)),
  -- (1009, 'Rahul', 'Sales', 35000, MOD(1009, 10)),
  -- (1010, 'Meghana', 'IT', 65000, MOD(1010, 10));
Query OK, 10 rows affected (0.01 sec)
Records: 10 Duplicates: 0 Warnings: 0

mysql> -- Display employees and hash buckets
mysql> SELECT *
mysql> FROM Employees
mysql> ORDER BY HashBucket;
+-----+-----+-----+-----+-----+
| EmployeeID | EmployeeName | Department | Salary | HashBucket |
+-----+-----+-----+-----+-----+
| 1010 | Meghana | IT | 65000.00 | 0 |
| 1001 | Ravi | IT | 45000.00 | 1 |
| 1002 | Anil | HR | 40000.00 | 2 |
| 1003 | Priya | Finance | 50000.00 | 3 |
| 1004 | Kiran | IT | 55000.00 | 4 |
| 1005 | Sneha | HR | 42000.00 | 5 |
| 1006 | Arjun | Sales | 38000.00 | 6 |
| 1007 | Vijay | IT | 60000.00 | 7 |
| 1008 | Pooja | Finance | 52000.00 | 8 |
| 1009 | Rahul | Sales | 35000.00 | 9 |
+-----+-----+-----+-----+-----+
10 rows in set (0.00 sec)

```



```
mysql>
mysql> -- Search employee using hash bucket
mysql> SELECT *
  -> FROM Employees
  -> WHERE HashBucket = MOD(1004, 10)
  -> AND EmployeeID = 1004;
+-----+-----+-----+-----+-----+
| EmployeeID | EmployeeName | Department | Salary | HashBucket |
+-----+-----+-----+-----+-----+
| 1004 | Kiran | IT | 55000.00 | 4 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Create index on EmployeeID
mysql> CREATE INDEX idx_employee_id
  -> ON Employees(EmployeeID);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Create index on HashBucket
mysql> CREATE INDEX idx_hash_bucket
  -> ON Employees(HashBucket);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Search employee
mysql> SELECT *
  -> FROM Employees
  -> WHERE EmployeeID = 1007;
+-----+-----+-----+-----+-----+
| EmployeeID | EmployeeName | Department | Salary | HashBucket |
+-----+-----+-----+-----+-----+
| 1007 | Vijay | IT | 60000.00 | 7 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Insert new employee
mysql> INSERT INTO Employees
  -> VALUES
  -> (1011, 'Suresh', 'IT', 48000, MOD(1011, 10));
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Delete employee
mysql> DELETE FROM Employees
  -> WHERE EmployeeID = 1011;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Display bucket distribution
mysql> SELECT HashBucket, COUNT(*) AS Employees
  -> FROM Employees
  -> GROUP BY HashBucket
  -> ORDER BY HashBucket;
+-----+-----+
| HashBucket | Employees |
+-----+-----+
| 0 | 1 |
| 1 | 1 |
| 2 | 1 |
| 3 | 1 |
| 4 | 1 |
| 5 | 1 |
| 6 | 1 |
| 7 | 1 |
| 8 | 1 |
| 9 | 1 |
+-----+-----+
10 rows in set (0.00 sec)

mysql> |
```

6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

An airline reservation system has two different requirements:

1. Point query using Flight Number.
2. Range query using Flight Date.

Since these operations have different characteristics, a combination of Hash Index + B+ Tree is recommended.

1. Hash Index on Flight Number

Create:

Hash Index

FlightNumber → Reservation Records

Example:

```
SELECT * FROM Flight WHERE FlightNumber = 'AI202';
```

Hashing provides very fast average exact lookup.

2. B+ Tree on Flight Date

Create:

B+ Tree

FlightDate → Reservation Records

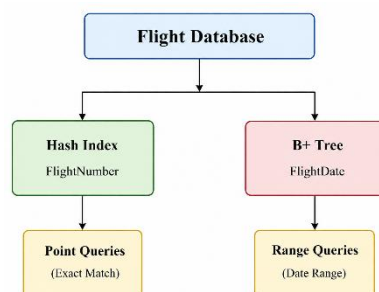
It supports queries such as:

```
SELECT * FROM Flight
```

```
WHERE FlightDate BETWEEN '2026-08-01'
```

```
AND '2026-08-15';
```

3. Combined Structure



4. Why Not Only Hashing?

Hashing is not suitable for date ranges because dates are not maintained in sorted order.

5. Why Not Only B+ Tree?

A B+ Tree can support both operations, so technically a single B+ Tree could be used. However, a hash index can provide very efficient exact lookup when FlightNumber searches dominate.

6. Trade-Off

Using two indexes increases:

- Storage.
- Insert/update cost.
- Index maintenance.

But it provides specialized performance for both types of queries.

Conclusion

A Hash Index on FlightNumber and B+ Tree on FlightDate is an effective combined indexing solution for an airline reservation system. If minimizing index maintenance is more important, a single composite B+ Tree can be considered.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli
mysql> -- =====
mysql> -- Q6: AIRLINE DATABASE
mysql> -- =====
mysql>
mysql> CREATE DATABASE AirlineDB;
ERROR 1007 (HY000): Can't create database 'airlinedb'; database exists
mysql> USE AirlineDB;
Database changed
mysql>
mysql> CREATE TABLE Flights (
  -> FlightID INT PRIMARY KEY,
  -> FlightNumber VARCHAR(20),
  -> FlightDate DATE,
  -> Source VARCHAR(50),
  -> Destination VARCHAR(50),
  -> AvailableSeats INT,
  -> Fare DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> -- Insert Flight records
mysql> INSERT INTO Flights
  -> VALUES
  -> (1, 'AI101', '2026-08-01', 'Delhi', 'Mumbai', 50, 5500),
  -> (2, 'AI102', '2026-08-02', 'Mumbai', 'Delhi', 40, 6000),
  -> (3, 'AI103', '2026-08-05', 'Hyderabad', 'Delhi', 30, 5000),
  -> (4, 'AI104', '2026-08-07', 'Chennai', 'Mumbai', 45, 4500),
  -> (5, 'AI105', '2026-08-10', 'Delhi', 'Hyderabad', 25, 5200),
  -> (6, 'AI106', '2026-08-12', 'Mumbai', 'Chennai', 35, 4800);
Query OK, 6 rows affected (0.01 sec)
Records: 6 Duplicates: 0 Warnings: 0

mysql> -- Index for FlightNumber
mysql> CREATE INDEX idx_flight_number
  -> ON Flights(FlightNumber);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> -- Index for FlightDate
mysql> CREATE INDEX idx_flight_date
  -> ON Flights(FlightDate);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> -- Point query
mysql> SELECT *
  -> FROM Flights
  -> WHERE FlightNumber = 'AI103';
+----+-----+-----+-----+-----+-----+-----+
| FlightID | FlightNumber | FlightDate | Source | Destination | AvailableSeats | Fare |
+----+-----+-----+-----+-----+-----+-----+
| 3 | AI103 | 2026-08-05 | Hyderabad | Delhi | 30 | 5000.00 |
+----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> -- Date range query
mysql> SELECT *
  -> FROM Flights
  -> WHERE FlightDate BETWEEN '2026-08-01'
  -> AND '2026-08-10';
+----+-----+-----+-----+-----+-----+-----+
| FlightID | FlightNumber | FlightDate | Source | Destination | AvailableSeats | Fare |
+----+-----+-----+-----+-----+-----+-----+
| 1 | AI101 | 2026-08-01 | Delhi | Mumbai | 50 | 5500.00 |
| 2 | AI102 | 2026-08-02 | Mumbai | Delhi | 40 | 6000.00 |
| 3 | AI103 | 2026-08-05 | Hyderabad | Delhi | 30 | 5000.00 |
| 4 | AI104 | 2026-08-07 | Chennai | Mumbai | 45 | 4500.00 |
| 5 | AI105 | 2026-08-10 | Delhi | Hyderabad | 25 | 5200.00 |
+----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql>
mysql> -- Combined search
mysql> SELECT *
  -> FROM Flights
  -> WHERE FlightNumber = 'AI103'
  -> AND FlightDate = '2026-08-05';
```

FlightID	FlightNumber	FlightDate	Source	Destination	AvailableSeats	Fare
3	AI103	2026-08-05	Hyderabad	Delhi	30	5000.00

1 row in set (0.00 sec)

```
mysql>
mysql> -- Check execution plan
mysql> EXPLAIN
  -> SELECT *
  -> FROM Flights
  -> WHERE FlightDate BETWEEN '2026-08-01'
  -> AND '2026-08-10';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Flights	NULL	range	idx_flight_date	idx_flight_date	4	NULL	5	100.00	Using index condition

1 row in set, 1 warning (0.00 sec)

```
mysql> |
```

7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Recommended Strategy: Heap File + Secondary Composite B+ Tree

A university database contains student information such as:

StudentID

Name

Department

CGPA

Year

The system needs fast queries based on Department and CGPA range.

1. File Organization

Use a Heap File to store student records.

Advantages:

- Fast insertion.
- No physical sorting required.
- Easy record deletion.
- Suitable for dynamic student records.

2. Secondary Index

Create a composite B+ Tree:

B+ Tree (Department, CGPA)

The first key is Department, followed by CGPA.

3. Example Query

```
SELECT * FROM Student WHERE Department = 'CSE' AND CGPA BETWEEN 8.0 AND 9.5;
```

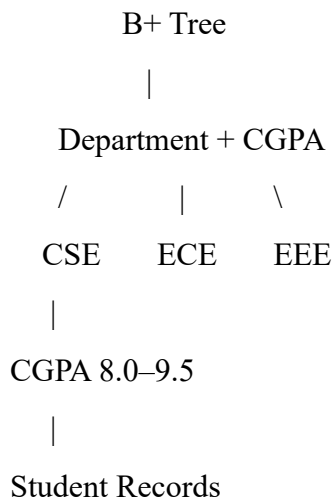
The composite B+ Tree index is created on (Department, CGPA). When this query is executed, the index first searches for the CSE department. After locating the CSE entries, it performs a range scan on the CGPA values and retrieves students whose CGPA is between 8.0 and 9.5. This avoids scanning the complete student table and improves query performance.

Example:

- Department = CSE
- $CGPA \geq 8.0$

- $CGPA \leq 9.5$
- Only matching student records are retrieved.

4. Index Structure



5. Why B+ Tree?

It supports:

- Equality search.
- Range search.
- Ordered traversal.
- Efficient disk access.
- Dynamic insertion and deletion.

6. Additional Index

If StudentID searches are extremely frequent, another B+ Tree or primary index can be maintained on StudentID.

7. Trade-Off

Additional indexes increase storage and update costs, but improve query response time.

Conclusion

The best design is a Heap File for student records with a secondary composite B+ Tree on (Department, CGPA). This provides efficient department filtering and CGPA range retrieval while maintaining flexible storage.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli

mysql> -- =====
mysql> -- Q7: UNIVERSITY DATABASE
mysql> -- =====
mysql> CREATE DATABASE UniversityDB;
ERROR 1007 (HY000): Can't create database 'universitydb'; database exists
mysql> USE UniversityDB;
Database changed
mysql> CREATE TABLE Students (
  -> StudentID INT PRIMARY KEY,
  -> StudentName VARCHAR(100),
  -> Department VARCHAR(50),
  -> CGPA DECIMAL(3,2),
  -> YearOfStudy INT
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> -- Insert student records
mysql> INSERT INTO Students
  -> VALUES
  -> (101, 'Rahul', 'CSE', 8.75, 3),
  -> (102, 'Priya', 'ECE', 9.10, 4),
  -> (103, 'Anil', 'CSE', 7.85, 2),
  -> (104, 'Sneha', 'EEE', 8.50, 3),
  -> (105, 'Kiran', 'CSE', 9.25, 4),
  -> (106, 'Pooja', 'ECE', 8.20, 2),
  -> (107, 'Arjun', 'CSE', 8.90, 3),
  -> (108, 'Vijay', 'EEE', 7.75, 2),
  -> (109, 'Meghana', 'CSE', 9.50, 4);
Query OK, 9 rows affected (0.01 sec)
Records: 9 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Display students
mysql> SELECT * FROM Students;
+-----+-----+-----+-----+-----+
| StudentID | StudentName | Department | CGPA | YearOfStudy |
+-----+-----+-----+-----+-----+
| 101 | Rahul | CSE | 8.75 | 3 |
| 102 | Priya | ECE | 9.10 | 4 |
| 103 | Anil | CSE | 7.85 | 2 |
| 104 | Sneha | EEE | 8.50 | 3 |
| 105 | Kiran | CSE | 9.25 | 4 |
| 106 | Pooja | ECE | 8.20 | 2 |
| 107 | Arjun | CSE | 8.90 | 3 |
| 108 | Vijay | EEE | 7.75 | 2 |
| 109 | Meghana | CSE | 9.50 | 4 |
+-----+-----+-----+-----+-----+
9 rows in set (0.00 sec)

mysql>
mysql> -- Composite index
mysql> CREATE INDEX idx_department_cgpa
  -> ON Students(Department, CGPA);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Search by department
mysql> SELECT *
  -> FROM Students
  -> WHERE Department = 'CSE';
+-----+-----+-----+-----+-----+
| StudentID | StudentName | Department | CGPA | YearOfStudy |
+-----+-----+-----+-----+-----+
| 103 | Anil | CSE | 7.85 | 2 |
| 101 | Rahul | CSE | 8.75 | 3 |
| 107 | Arjun | CSE | 8.90 | 3 |
| 105 | Kiran | CSE | 9.25 | 4 |
| 109 | Meghana | CSE | 9.50 | 4 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- Search by department and CGPA range
mysql> SELECT *
  -> FROM Students
  -> WHERE Department = 'CSE'
  -> AND CGPA BETWEEN 8.0 AND 9.5;
+-----+-----+-----+-----+-----+
| StudentID | StudentName | Department | CGPA | YearOfStudy |
+-----+-----+-----+-----+-----+
| 101 | Rahul | CSE | 8.75 | 3 |
| 107 | Arjun | CSE | 8.90 | 3 |
| 105 | Kiran | CSE | 9.25 | 4 |
| 109 | Meghana | CSE | 9.50 | 4 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> -- Display students in CGPA order
mysql> SELECT *
  -> FROM Students
  -> WHERE Department = 'CSE'
  -> ORDER BY CGPA;
+-----+-----+-----+-----+-----+
| StudentID | StudentName | Department | CGPA | YearOfStudy |
+-----+-----+-----+-----+-----+
| 103 | Anil | CSE | 7.85 | 2 |
| 101 | Rahul | CSE | 8.75 | 3 |
| 107 | Arjun | CSE | 8.90 | 3 |
| 105 | Kiran | CSE | 9.25 | 4 |
| 109 | Meghana | CSE | 9.50 | 4 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> -- Check index usage
mysql> EXPLAIN
  -> SELECT *
  -> FROM Students
  -> WHERE Department = 'CSE'
  -> AND CGPA BETWEEN 8.0 AND 9.5;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Students | NULL | range | idx_department_cgpa | idx_department_cgpa | 206 | NULL | 4 | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
```


8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Given Data

- Number of records = 100,000
- Record size = 50 bytes
- Block size = 4 KB
- 4 KB = 4096 bytes

Step 1: Records per Block

$$\begin{aligned}\text{Records per block} &= \left\lfloor \frac{4096}{50} \right\rfloor \\ &= 81\end{aligned}$$

Therefore, each block can contain 81 records.

Step 2: Number of Blocks

$$\begin{aligned}\text{Blocks} &= \left\lceil \frac{100000}{81} \right\rceil \\ &= \boxed{1235 \text{ blocks}}\end{aligned}$$

Step 3: Total Record Storage

$$100000 \times 50 = 5,000,000 \text{ bytes}$$

This is approximately:

$$\frac{5,000,000}{1024^2} \approx 4.77 \text{ MB}$$

Step 4: Allocated Storage

$$1235 \times 4096 = 5,058,560 \text{ bytes}$$

Step 5: Unused Space

$$5,058,560 - 5,000,000 = 58,560 \text{ bytes}$$

Step 6: Utilization

$$\begin{aligned}\text{Utilization} &= \frac{5,000,000}{5,058,560} \times 100 \\ &= \boxed{98.84\%}\end{aligned}$$

Thus, approximately 98.84% of the allocated block space is used.

Comparison

File Organization	Approx.Data Blocks	Utilization	Important Characteristic
Heap	1235	98.84%	Unordered
Sorted	1235	98.84%	Ordered
Hashed	≈1235 ideally	≈98.84% ideally	Hash-based placement

Heap File:

Records are placed wherever space is available.

Advantage: Fast insertion.

Disadvantage: Searching may require a full scan.

Sorted File:

Records are maintained in sorted order.

Advantage: Excellent sequential and range access.

Disadvantage: Insertions can be expensive because records may need to be moved.

Hashed File:

Records are placed using a hash function.

Advantage: Very fast equality searches.

Disadvantage: Collisions can create overflow blocks and reduce actual utilization.

Conclusion

All three can theoretically achieve approximately 98.84% utilization when records are packed efficiently. However, actual hashed utilization may be lower due to collisions, while sorted files may require extra free space to accommodate future insertions.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli  X  +  v

mysql> -- =====
mysql> -- Q8: DISK BLOCK UTILIZATION
mysql> -- =====
mysql>
mysql> CREATE DATABASE BlockDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE BlockDB;
Database changed
mysql>
mysql> CREATE TABLE BlockCalculation (
  ->   NumberOfRecords INT,
  ->   RecordSize INT,
  ->   BlockSize INT
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO BlockCalculation
  -> VALUES (100000, 50, 4096);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> -- Display input
mysql> SELECT * FROM BlockCalculation;
+-----+-----+-----+
| NumberOfRecords | RecordSize | BlockSize |
+-----+-----+-----+
|          100000 |          50 |         4096 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Records per block
mysql> SELECT
  ->   FLOOR(BlockSize / RecordSize) AS RecordsPerBlock
  -> FROM BlockCalculation;
+-----+
| RecordsPerBlock |
+-----+
|              81 |
+-----+
1 row in set (0.01 sec)

mysql>
mysql> -- Number of blocks required
mysql> SELECT
  ->   CEIL(NumberOfRecords /
  ->        FLOOR(BlockSize / RecordSize)) AS BlocksRequired
  -> FROM BlockCalculation;
+-----+
| BlocksRequired |
+-----+
|             1235 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Total record storage
mysql> SELECT
  ->   NumberOfRecords * RecordSize AS TotalRecordBytes
  -> FROM BlockCalculation;
+-----+
| TotalRecordBytes |
+-----+
|          5000000 |
+-----+
1 row in set (0.00 sec)
```

```
mysql>
mysql> -- Allocated block storage
mysql> SELECT
  ->   CEIL(NumberOfRecords /
  ->        FLOOR(BlockSize / RecordSize))
  ->   * BlockSize AS AllocatedBytes
  -> FROM BlockCalculation;
+-----+
| AllocatedBytes |
+-----+
|          5058560 |
+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Calculate utilization
mysql> SELECT
  ->   ROUND(
  ->     (NumberOfRecords * RecordSize) /
  ->     (
  ->       CEIL(NumberOfRecords /
  ->            FLOOR(BlockSize / RecordSize))
  ->       * BlockSize
  ->     ) * 100,
  ->     2
  -> ) AS BlockUtilizationPercentage
  -> FROM BlockCalculation;
+-----+
| BlockUtilizationPercentage |
+-----+
|              98.84 |
+-----+
1 row in set (0.00 sec)

mysql> |
```

9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Recommended Strategy: Composite B+ Tree Indexing with Partitioning

A social media platform with 500 million posts represents a very large database. It needs efficient access using:

- user_id
- timestamp
- hashtag

A single index on all three attributes may not be suitable for every query. Therefore, a combination of composite indexes and partitioning is recommended.

1. User-Based Composite Index

Create: B+ Tree (user_id, timestamp)

This supports: Find all posts by a user

and:

Find a user's posts during a specific period

Example:

WHERE user_id = 10025

AND timestamp BETWEEN T1 AND T2;

2. Hashtag-Based Composite Index

Create: B+ Tree (hashtag, timestamp)

This supports: Find all #AI posts

and:

Find #AI posts created during the last 24 hours

3. Timestamp Partitioning

With 500 million records, partitioning is highly beneficial.

For example:

Posts

|

+-- January 2026

+-- February 2026

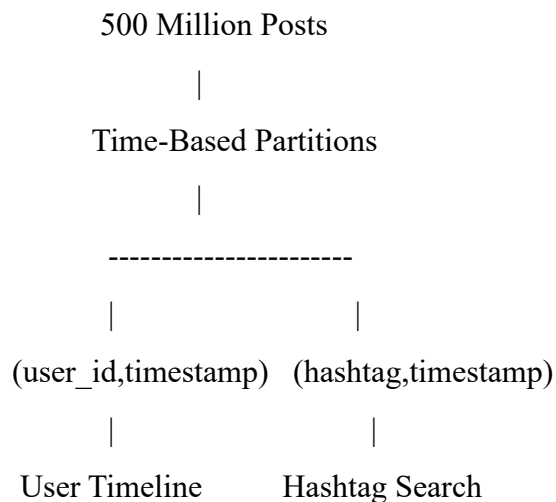
+-- March 2026

+-- April 2026

+-- ...

When a query requests recent posts, the DBMS can avoid scanning old partitions.

4. Proposed Architecture



5. Advantages

- Fast user timeline retrieval.
- Efficient hashtag searches.
- Efficient time-range queries.
- Partition pruning reduces unnecessary disk I/O.
- B+ Tree supports ordered timestamp access.
- Scales better for continuously growing data.

6. Important Trade-Off

Creating too many indexes causes:

- Higher storage usage.
- Higher insertion cost.
- Higher update/delete cost.
- More index maintenance.

Therefore, indexes should be created only for frequently executed queries.

Conclusion

For 500 million social media posts, use time-based partitioning with composite B+ Tree indexes (user_id, timestamp) and (hashtag, timestamp). This design provides scalability and efficient retrieval while controlling the cost of indexing a very large dataset.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli
mysql> --
mysql> -- Q9: SOCIAL MEDIA DATABASE
mysql>
mysql> CREATE DATABASE SocialMediaDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE SocialMediaDB;
Database changed
mysql>
mysql> CREATE TABLE Posts (
  -> PostID BIGINT PRIMARY KEY,
  -> UserID BIGINT NOT NULL,
  -> PostText VARCHAR(500),
  -> Hashtag VARCHAR(100),
  -> PostTimestamp DATETIME NOT NULL
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> -- Insert sample posts
mysql> INSERT INTO Posts
  -> VALUES
  -> (1, 101, 'Learning AI', '#AI', '2026-08-01 10:00:00'),
  -> (2, 101, 'Database project', '#DBMS', '2026-08-02 11:00:00'),
  -> (3, 102, 'Machine Learning', '#ML', '2026-08-03 12:00:00'),
  -> (4, 103, 'Artificial Intelligence', '#AI', '2026-08-04 13:00:00'),
  -> (5, 101, 'SQL indexing', '#DBMS', '2026-08-05 14:00:00'),
  -> (6, 104, 'AI project', '#AI', '2026-08-06 15:00:00'),
  -> (7, 102, 'Data Science', '#DS', '2026-08-07 16:00:00');
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Composite index 1
mysql> CREATE INDEX idx_user_timestamp
  -> ON Posts(UserID, PostTimestamp);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Composite index 2
mysql> CREATE INDEX idx_hashtag_timestamp
  -> ON Posts(Hashtag, PostTimestamp);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Find all posts by a user
mysql> SELECT *
  -> FROM Posts
  -> WHERE UserID = 101;
+-----+-----+-----+-----+-----+
| PostID | UserID | PostText | Hashtag | PostTimestamp |
+-----+-----+-----+-----+-----+
| 1 | 101 | Learning AI | #AI | 2026-08-01 10:00:00 |
| 2 | 101 | Database project | #DBMS | 2026-08-02 11:00:00 |
| 5 | 101 | SQL indexing | #DBMS | 2026-08-05 14:00:00 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Find user's posts in a date/time range
mysql> SELECT *
  -> FROM Posts
  -> WHERE UserID = 101
  -> AND PostTimestamp BETWEEN
  -> '2026-08-01 00:00:00'
  -> AND '2026-08-31 23:59:59';
+-----+-----+-----+-----+-----+
| PostID | UserID | PostText | Hashtag | PostTimestamp |
+-----+-----+-----+-----+-----+
| 1 | 101 | Learning AI | #AI | 2026-08-01 10:00:00 |
| 2 | 101 | Database project | #DBMS | 2026-08-02 11:00:00 |
| 5 | 101 | SQL indexing | #DBMS | 2026-08-05 14:00:00 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Find posts using hashtag
mysql> SELECT *
  -> FROM Posts
  -> WHERE Hashtag = '#AI';
+-----+-----+-----+-----+-----+
| PostID | UserID | PostText | Hashtag | PostTimestamp |
+-----+-----+-----+-----+-----+
| 1 | 101 | Learning AI | #AI | 2026-08-01 10:00:00 |
| 4 | 103 | Artificial Intelligence | #AI | 2026-08-04 13:00:00 |
| 6 | 104 | AI project | #AI | 2026-08-06 15:00:00 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Find hashtag posts in a time range
mysql> SELECT *
  -> FROM Posts
  -> WHERE Hashtag = '#AI'
  -> AND PostTimestamp BETWEEN
  -> '2026-08-01 00:00:00'
  -> AND '2026-08-31 23:59:59';
+-----+-----+-----+-----+-----+
| PostID | UserID | PostText | Hashtag | PostTimestamp |
+-----+-----+-----+-----+-----+
| 1 | 101 | Learning AI | #AI | 2026-08-01 10:00:00 |
| 4 | 103 | Artificial Intelligence | #AI | 2026-08-04 13:00:00 |
| 6 | 104 | AI project | #AI | 2026-08-06 15:00:00 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> -- Check index usage
mysql> EXPLAIN
  -> SELECT *
  -> FROM Posts
  -> WHERE UserID = 101
  -> AND PostTimestamp BETWEEN
  -> '2026-08-01 00:00:00'
  -> AND '2026-08-31 23:59:59';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Posts | NULL | range | idx_user_timestamp | idx_user_timestamp | 13 | NULL | 3 | 100.00 | Using index condition |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
```

10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

A supermarket billing system handles approximately 1 million transactions per day. The system needs fast transaction insertion and retrieval.

The three possible file organizations are:

1. Heap File
2. Sorted File
3. Hashed File

1. Heap File

Records are stored without any particular order.

Insert

A new transaction can be added to a free location.

Average cost: approximately $O(1)$

Search

Without an index, the system may need to scan the file.

Cost: $O(N)$

For 1 million records, this can be expensive.

Delete

Finding the record may require a scan.

Cost: $O(N)$ without an index.

Advantage

Excellent for high-volume transaction insertion.

2. Sorted File

Transactions are stored according to a key such as:

TransactionID or TransactionDate

Search

Binary search can be used.

Cost: approximately $O(\log N)$ for locating records.

Range Search

Very efficient.

For example:

Find all transactions between 10:00 AM and 12:00 PM can be performed efficiently because records are ordered.

Insert

Insertion can be expensive because records may need to be shifted.

Cost: approximately $O(N)$ in the worst case.

Advantage

Excellent for reporting and range queries.

3. Hashed File

A hash function maps each transaction to a bucket.

For example:

$$h(\text{TransactionID}) = \text{TransactionID} \text{ MOD } M$$

Insert

Average: $O(1)$

Exact Search

Average: $O(1)$

Therefore, searching for:

TransactionID = 100245 is extremely fast.

Range Search

Poor because hash values do not preserve the order of TransactionID or Date

4. Comparison

Operation	Heap	Sorted	Hashed
Insert	$O(1)$ average	$O(N)$	$O(1)$ average
Exact Search	$O(N)$	$O(\log N)$	$O(1)$ average
Delete	$O(N)$	$O(N)$	$O(1)$ average
Range Search	$O(N)$	$O(\log N + k)$	Poor
Sequential Access	Good	Excellent	Poor
Overflow	No	No	Possible

Here, k represents the number of records returned by a range query.

5. Best Choice

If the supermarket mainly performs:

Find transaction using TransactionID , then Hashed File Organization is the best choice.

If the system mainly performs:

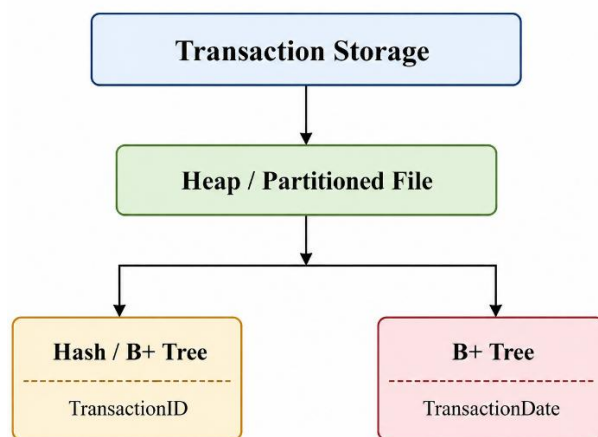
Daily sales reports

Hourly transaction reports

Transactions between two dates, then a Sorted File or B+ Tree index is more suitable.

Practical Recommendation

For a real supermarket system handling 1 million transactions daily:



This hybrid approach gives:

- Fast transaction insertion.
- Fast exact transaction lookup.
- Efficient date-range reports.
- Better scalability.

Conclusion

For pure exact transaction lookup, Hashed File is the best because of its $O(1)$ average search and insertion. However, for a practical supermarket billing system with both billing and reporting requirements, a Heap/partitioned file with Hash or B+ Tree indexes provides the best overall performance and scalability.

SQL IMPLEMENTATION

```
MySQL 8.0 Command Line Cli × + ▾

mysql> -- =====
mysql> -- Q10: SUPERMARKET BILLING DATABASE
mysql> -- =====
mysql>
mysql> CREATE DATABASE SupermarketDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE SupermarketDB;
Database changed
mysql>
mysql>
mysql> -- =====
mysql> -- HEAP FILE SIMULATION
mysql> -- =====
mysql>
mysql> CREATE TABLE HeapTransactions (
  -> TransactionID INT PRIMARY KEY,
  -> TransactionDate DATETIME,
  -> CustomerName VARCHAR(100),
  -> Amount DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO HeapTransactions
  -> VALUES
  -> (1001, '2026-08-01 10:00:00', 'Ravi', 500.00),
  -> (1002, '2026-08-01 10:15:00', 'Anil', 750.00),
  -> (1003, '2026-08-01 11:00:00', 'Priya', 1200.00),
  -> (1004, '2026-08-01 12:00:00', 'Kiran', 450.00),
  -> (1005, '2026-08-01 13:00:00', 'Sneha', 900.00);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> SELECT * FROM HeapTransactions;
+-----+-----+-----+-----+
| TransactionID | TransactionDate | CustomerName | Amount |
+-----+-----+-----+-----+
| 1001 | 2026-08-01 10:00:00 | Ravi | 500.00 |
| 1002 | 2026-08-01 10:15:00 | Anil | 750.00 |
| 1003 | 2026-08-01 11:00:00 | Priya | 1200.00 |
| 1004 | 2026-08-01 12:00:00 | Kiran | 450.00 |
| 1005 | 2026-08-01 13:00:00 | Sneha | 900.00 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- SORTED FILE SIMULATION
mysql> -- =====
mysql>
mysql> CREATE TABLE SortedTransactions (
  -> TransactionID INT PRIMARY KEY,
  -> TransactionDate DATETIME,
  -> CustomerName VARCHAR(100),
  -> Amount DECIMAL(10,2)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO SortedTransactions
  -> VALUES
  -> (1001, '2026-08-01 10:00:00', 'Ravi', 500.00),
  -> (1002, '2026-08-01 10:15:00', 'Anil', 750.00),
  -> (1003, '2026-08-01 11:00:00', 'Priya', 1200.00),
  -> (1004, '2026-08-01 12:00:00', 'Kiran', 450.00),
  -> (1005, '2026-08-01 13:00:00', 'Sneha', 900.00);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```



```
mysql> INSERT INTO SortedTransactions
-> VALUES
-> (1001, '2026-08-01 10:00:00', 'Ravi', 500.00),
-> (1002, '2026-08-01 10:15:00', 'Anil', 750.00),
-> (1003, '2026-08-01 11:00:00', 'Priya', 1200.00),
-> (1004, '2026-08-01 12:00:00', 'Kiran', 450.00),
-> (1005, '2026-08-01 13:00:00', 'Sneha', 900.00);
```

Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

```
mysql>
mysql> SELECT *
-> FROM SortedTransactions
-> ORDER BY TransactionID;
```

TransactionID	TransactionDate	CustomerName	Amount
1001	2026-08-01 10:00:00	Ravi	500.00
1002	2026-08-01 10:15:00	Anil	750.00
1003	2026-08-01 11:00:00	Priya	1200.00
1004	2026-08-01 12:00:00	Kiran	450.00
1005	2026-08-01 13:00:00	Sneha	900.00

5 rows in set (0.00 sec)

```
mysql>
mysql>
mysql> -- =====
mysql> -- HASHED FILE SIMULATION
mysql> -- =====
mysql>
mysql> CREATE TABLE HashedTransactions (
-> TransactionID INT PRIMARY KEY,
-> TransactionDate DATETIME,
-> CustomerName VARCHAR(100),
-> Amount DECIMAL(10,2),
-> HashBucket INT
-> );
```

Query OK, 0 rows affected (0.03 sec)

```
mysql>
mysql> INSERT INTO HashedTransactions
-> VALUES
-> (1001, '2026-08-01 10:00:00', 'Ravi', 500.00, MOD(1001,10)),
-> (1002, '2026-08-01 10:15:00', 'Anil', 750.00, MOD(1002,10)),
-> (1003, '2026-08-01 11:00:00', 'Priya', 1200.00, MOD(1003,10)),
-> (1004, '2026-08-01 12:00:00', 'Kiran', 450.00, MOD(1004,10)),
-> (1005, '2026-08-01 13:00:00', 'Sneha', 900.00, MOD(1005,10));
```

Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

```
mysql>
mysql> SELECT *
-> FROM HashedTransactions
-> ORDER BY HashBucket;
```

TransactionID	TransactionDate	CustomerName	Amount	HashBucket
1001	2026-08-01 10:00:00	Ravi	500.00	1
1002	2026-08-01 10:15:00	Anil	750.00	2
1003	2026-08-01 11:00:00	Priya	1200.00	3
1004	2026-08-01 12:00:00	Kiran	450.00	4
1005	2026-08-01 13:00:00	Sneha	900.00	5

5 rows in set (0.00 sec)

```
MySQL 8.0 Command Line Cli  X + v

mysql> -- =====
mysql> -- SEARCH COMPARISON
mysql> -- =====
mysql>
mysql> -- Heap search
mysql> SELECT *
  -> FROM HeapTransactions
  -> WHERE TransactionID = 1003;
+-----+-----+-----+-----+
| TransactionID | TransactionDate | CustomerName | Amount |
+-----+-----+-----+-----+
|          1003 | 2026-08-01 11:00:00 | Priya        | 1200.00 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Sorted search
mysql> SELECT *
  -> FROM SortedTransactions
  -> WHERE TransactionID = 1003;
+-----+-----+-----+-----+
| TransactionID | TransactionDate | CustomerName | Amount |
+-----+-----+-----+-----+
|          1003 | 2026-08-01 11:00:00 | Priya        | 1200.00 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> -- Hashed search
mysql> SELECT *
  -> FROM HashedTransactions
  -> WHERE HashBucket = MOD(1003,10)
  -> AND TransactionID = 1003;
+-----+-----+-----+-----+-----+
| TransactionID | TransactionDate | CustomerName | Amount | HashBucket |
+-----+-----+-----+-----+-----+
|          1003 | 2026-08-01 11:00:00 | Priya        | 1200.00 |           3 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- RANGE QUERY COMPARISON
mysql> -- =====
mysql>
mysql> -- Heap
mysql> SELECT *
  -> FROM HeapTransactions
  -> WHERE TransactionDate BETWEEN
  -> '2026-08-01 10:00:00'
  -> AND '2026-08-01 12:00:00';
+-----+-----+-----+-----+
| TransactionID | TransactionDate | CustomerName | Amount |
+-----+-----+-----+-----+
|          1001 | 2026-08-01 10:00:00 | Ravi        | 500.00 |
|          1002 | 2026-08-01 10:15:00 | Anil        | 750.00 |
|          1003 | 2026-08-01 11:00:00 | Priya        | 1200.00 |
|          1004 | 2026-08-01 12:00:00 | Kiran        | 450.00 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
MySQL 8.0 Command Line Cli  X + v

mysql>
mysql> -- Hashed
mysql> SELECT *
  -> FROM HashedTransactions
  -> WHERE TransactionDate BETWEEN
  -> '2026-08-01 10:00:00'
  -> AND '2026-08-01 12:00:00';
+-----+-----+-----+-----+-----+
| TransactionID | TransactionDate | CustomerName | Amount | HashBucket |
+-----+-----+-----+-----+-----+
|          1001 | 2026-08-01 10:00:00 | Ravi        | 500.00 |           1 |
|          1002 | 2026-08-01 10:15:00 | Anil        | 750.00 |           2 |
|          1003 | 2026-08-01 11:00:00 | Priya        | 1200.00 |           3 |
|          1004 | 2026-08-01 12:00:00 | Kiran        | 450.00 |           4 |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql>
mysql> -- =====
mysql> -- CREATE INDEX FOR PERFORMANCE TEST
mysql> -- =====
mysql>
mysql> CREATE INDEX idx_sorted_date
  -> ON SortedTransactions(TransactionDate);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> CREATE INDEX idx_hashed_bucket
  -> ON HashedTransactions(HashBucket);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Execution plans
mysql>
mysql> EXPLAIN
  -> SELECT *
  -> FROM HeapTransactions
  -> WHERE TransactionID = 1003;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | HeapTransactions | NULL | const | PRIMARY | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql> EXPLAIN
  -> SELECT *
  -> FROM SortedTransactions
  -> WHERE TransactionID = 1003;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | SortedTransactions | NULL | const | PRIMARY | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql> EXPLAIN
  -> SELECT *
  -> FROM HashedTransactions
  -> WHERE HashBucket = MOD(1003,10)
  -> AND TransactionID = 1003;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | HashedTransactions | NULL | const | PRIMARY, idx_hashed_bucket | PRIMARY | 4 | const | 1 | 100.00 | NULL |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> |
```